

What the program is

The program that we created will allow you to execute BasicML programs. It will load a BasicML program into memory and follow the instructions. Then it will perform calculations using an accumulator. It will also have input, output, arithmetic operations, branching and program termination.

User stories

User story 1

As a computer science student, I want a program that will allow a BasicML program to load so that I can execute machine language instruction and see how the virtual machine works.

User story 2

As a computer science student, I want the program to process BasicML instructions and to show the results so that I can learn how the accumulator and memory work together.

Use cases

1. Load basicML File

Actor: Student

System:

- Student starts the program
- Program prompts for a file name
- Student enters the file name for the input
- The program loads the program into the memory locations that were allocated. In our program, it will be 0-99

Goal: Load basicML into memory

2. Validate BasicML File

Actor: Student

System:

- The program will read the file
- Then the program will check to see if the instructions are correct
- Then the program will verify if it will be able to fit into the memory

Goal: Verify that the file contains valid instructions

3. Execute Read instruction

Actor: Student

System:

- When the program comes across a read instruction it will prompt for user input
- The user will then enter input (a number)
- The program will then store that value into its memory

Goal: To get input from the user to store into its memory

4. Execute Write instruction

Actor: Student

System:

- When the program comes across a write instruction, it will get the value from its memory
- Then it will display the value

Goal: To display value from its current memory

5. Execute Load instruction

Actor: Student

System:

- When the system comes across a load instruction, it will get the value from its memory
- Places that value into the accumulator

Goal: To load a value from its memory into the accumulator

6. Execute Store instruction

Actor: Student

System:

- When the program comes across a store instruction, it will copy the accumulator value
- Then it will store it into its memory

Goal: The goal is to store the accumulator value into its memory

7. Execute Add instruction

Actor: Student

System:

- When the program comes across an add instruction, it will get the memory value
- Add that value into the accumulator

Goal: The goal here is to add the memory value to the accumulator

8. Execute Subtract instruction

Actor: Student

System:

- Program comes across subtract instruction, it will get memory value
- Subtract that value from the accumulator

Goal: The goal is to subtract the value from the accumulator

9. Execute Divide instruction

Actor: Student

System:

- When it comes across a divide instruction, it will get the memory value
- Then it will divide

Goal: The goal is to divide the accumulator by the memory value

10. Execute Multiple instruction

Actor: Student

System:

- When it comes across a multiply instruction, it will get the memory value
- Then it will multiply the values

Goal: Multiple the accumulator by the memory value

11. Execute Branch instruction

Actor: Student

System:

- When it comes across a branch instruction, it will read the location specified
- Then it jumps to that location
- Then it will execute from the location

Goal: The goal here is to move the execution to a different place

12. Execute Branchneg instruction

Actor: Student

System:

- When the program comes across a branchneg instruction, it will check the value from the accumulator
- If that value is less than 0, it will jump to the memory location
- If it's not less than 0, it will continue to the next instruction

Goal: The goal here is to branch to a specific location in memory if the accumulator is negative

13. Execute Branchzero instruction

Actor: Student

System:

- When it comes across a branchzero instruction, it will check the value stored inside the accumulator

- If it's zero, it will jump to the specified memory location

- If it's not zero, it will go forward to the next instruction

Goal: To branch into a specific location in the memory if the accumulator is zero

14. Execute Halt instruction

Actor: Student

System:

- When it comes across the halt instruction, it will terminate execution

- program will stop running

Goal: Stop the program